

In []:

```
In [1]: import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import os
import time

# 假设你的 QuantConv2d 等定义在 models.py 中
# 如果 models.py 也是你自己写的, 请确保里面的 tensor 也是创建在正确的 device 上
from models import * # =====
# 1. 设备配置 (MacBook 适配核心)
# =====
def get_device():
    if torch.backends.mps.is_available():
        return torch.device("mps") # Apple Silicon GPU 加速
    elif torch.cuda.is_available():
        return torch.device("cuda") # NVIDIA GPU
    else:
        return torch.device("cpu") # 传统 CPU

device = get_device()
print(f"当前运行设备: {device}")

# =====
# 2. 模型定义 (VGG16_Part1) - 已修改保留 BN
# =====
class VGG16_Part1(nn.Module):
    def __init__(self, num_classes=10):
        super(VGG16_Part1, self).__init__()

        self.features = nn.Sequential(
            # --- Block 1 ---
            QuantConv2d(3, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64)
            QuantConv2d(64, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64)
            nn.MaxPool2d(kernel_size=2, stride=2),

            # --- Block 2 ---
            QuantConv2d(64, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128)
            QuantConv2d(128, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128)
            nn.MaxPool2d(kernel_size=2, stride=2),

            # --- Block 3 ---
            QuantConv2d(128, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256)
            QuantConv2d(256, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256)
            QuantConv2d(256, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256)
            nn.MaxPool2d(kernel_size=2, stride=2),

            # --- Block 4 ---
            QuantConv2d(256, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512)
            QuantConv2d(512, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512)
            QuantConv2d(512, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512)
```

```

        nn.MaxPool2d(kernel_size=2, stride=2),

        # --- Block 5 (修改区域) ---
        # 原本的第一层: 512 -> 512
        QuantConv2d(512, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),

        # ===== PART 1 核心修改: 8x8 Squeezed Layer =====
        # 1. 适配层 (Adapter): 512 -> 8 (使用 1x1 卷积降维)
        QuantConv2d(512, 8, kernel_size=1),
        nn.BatchNorm2d(8),
        nn.ReLU(inplace=True),

        # 2. 目标层 (Target Layer): 8 -> 8 (3x3 卷积)
        # 【修改点】: 保留 BN
        QuantConv2d(8, 8, kernel_size=3, padding=1, bias=False), # 有BN这
        nn.BatchNorm2d(8), # <--- 这里的 BN 被保留了
        nn.ReLU(inplace=True),

        # 3. 恢复层 (Expand): 8 -> 512 (使用 1x1 卷积升维)
        QuantConv2d(8, 512, kernel_size=1),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        # =====

        # Block 5 剩余部分
        QuantConv2d(512, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512),
        nn.MaxPool2d(kernel_size=2, stride=2),
        nn.AvgPool2d(kernel_size=1, stride=1),
    )

    self.classifier = nn.Linear(512, num_classes)

    def forward(self, x):
        x = self.features(x)
        x = x.view(x.size(0), -1)
        x = self.classifier(x)
        return x

# 实例化并移至对应设备 (MPS or CPU)
model_part1 = VGG16_Part1().to(device)
print("VGG16_Part1 模型构建完成。目标 8x8 层已保留 BN。")

# =====
# 3. 权重加载函数 (适配 Mac)
# =====
def load_pretrained_weights(model, pretrained_path):
    if os.path.isfile(pretrained_path):
        print(f"=> loading checkpoint '{pretrained_path}'")

        # 【关键修改】: map_location 确保在 Mac 上能加载 CUDA 训练的权重
        checkpoint = torch.load(pretrained_path, map_location=device)

        pretrained_dict = checkpoint['state_dict']
        model_dict = model.state_dict()

        # 过滤不匹配层

```

```

        pretrained_dict = {k: v for k, v in pretrained_dict.items()
                             if k in model_dict and v.shape == model_dict[k].shape}

    model_dict.update(pretrained_dict)
    model.load_state_dict(model_dict)

    print(f"已加载预训练权重。忽略了 {len(model.state_dict()) - len(pretrained_dict)} 个参数")
else:
    print(f"在 '{pretrained_path}' 未找到 checkpoint")

PRETRAINED_PATH = "result/VGG16_quant/model_best.pth.tar"
# 即使文件不存在也不报错，方便你直接运行代码测试逻辑
if os.path.exists(PRETRAINED_PATH):
    load_pretrained_weights(model_part1, PRETRAINED_PATH)
else:
    print(f"提示: '{PRETRAINED_PATH}' 不存在，跳过加载权重，使用随机初始化。")

# =====
# 4. 数据加载 (适配 Mac)
# =====
normalize = transforms.Normalize(mean=[0.491, 0.482, 0.447], std=[0.247, 0.242, 0.245])

train_dataset = torchvision.datasets.CIFAR10(
    root='./data', train=True, download=True,
    transform=transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        normalize,
    ]))

# Mac 上 num_workers 设置过大可能导致多进程错误，通常 0（主进程）或 2 比较稳妥
trainloader = torch.utils.data.DataLoader(
    train_dataset, batch_size=128, shuffle=True, num_workers=2
)

test_dataset = torchvision.datasets.CIFAR10(
    root='./data', train=False, download=True,
    transform=transforms.Compose([
        transforms.ToTensor(),
        normalize,
    ]))

testloader = torch.utils.data.DataLoader(
    test_dataset, batch_size=128, shuffle=False, num_workers=2
)

# =====
# 5. 训练循环 (适配 MPS/CPU)
# =====
criterion = nn.CrossEntropyLoss().to(device)
optimizer = optim.SGD(model_part1.parameters(), lr=0.01, momentum=0.9, weight_decay=1e-4)
scheduler = optim.lr_scheduler.MultiStepLR(optimizer, milestones=[20, 40], gamma=0.1)

def fine_tune_advanced(model, train_loader, test_loader, epochs=50):
    best_acc = 0

```

```

model.to(device) # 确保模型在 MPS/CPU

for epoch in range(epochs):
    # --- Training ---
    model.train()
    for i, (inputs, targets) in enumerate(train_loader):
        # 将数据移动到 Mac 支持的设备
        inputs, targets = inputs.to(device), targets.to(device)

        optimizer.zero_grad()
        output = model(inputs)
        loss = criterion(output, targets)
        loss.backward()
        optimizer.step()

    # 更新学习率
    scheduler.step()
    current_lr = scheduler.get_last_lr()[0]

    # --- Validation ---
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for inputs, targets in test_loader:
            inputs, targets = inputs.to(device), targets.to(device)
            output = model(inputs)
            _, predicted = output.max(1)
            total += targets.size(0)
            correct += predicted.eq(targets).sum().item()

    acc = 100. * correct / total

    print(f"Epoch [{epoch+1}/{epochs}] (LR: {current_lr:.5f}) -> Test Acc: {acc:.2f}%")

    # 保存最佳模型 (保存到 CPU 格式, 兼容性最好)
    if acc > best_acc:
        best_acc = acc
        # 确保文件夹存在
        if not os.path.exists("result"):
            os.makedirs("result")
        torch.save({'state_dict': model.state_dict()}, "result/vgg_16.pth")
        print(f"    New Best found: {best_acc:.2f}% (Saved)")

    print(f" Advanced Fine-tuning 完成! 最终最佳精度: {best_acc:.2f}%")
    return best_acc

# 开始训练
fine_tune_advanced(model_part1, trainloader, testloader, epochs=50)

```

当前运行设备：mps

VGG16_Part1 模型构建完成。目标 8x8 层已保留 BN。

=> loading checkpoint 'result/VGG16_quant/model_best.pth.tar'

已加载预训练权重。忽略了 30 个不匹配的参数层。

Epoch [1/50] (LR: 0.01000) -> Test Accuracy: 87.64%

New Best found: 87.64% (Saved)

Epoch [2/50] (LR: 0.01000) -> Test Accuracy: 89.74%

New Best found: 89.74% (Saved)

Epoch [3/50] (LR: 0.01000) -> Test Accuracy: 88.58%

Epoch [4/50] (LR: 0.01000) -> Test Accuracy: 88.74%

Epoch [5/50] (LR: 0.01000) -> Test Accuracy: 89.67%

Epoch [6/50] (LR: 0.01000) -> Test Accuracy: 87.74%

Epoch [7/50] (LR: 0.01000) -> Test Accuracy: 89.91%

New Best found: 89.91% (Saved)

Epoch [8/50] (LR: 0.01000) -> Test Accuracy: 89.45%

Epoch [9/50] (LR: 0.01000) -> Test Accuracy: 89.57%

Epoch [10/50] (LR: 0.01000) -> Test Accuracy: 89.10%

Epoch [11/50] (LR: 0.01000) -> Test Accuracy: 89.77%

Epoch [12/50] (LR: 0.01000) -> Test Accuracy: 89.51%

Epoch [13/50] (LR: 0.01000) -> Test Accuracy: 90.27%

New Best found: 90.27% (Saved)

Epoch [14/50] (LR: 0.01000) -> Test Accuracy: 89.51%

Epoch [15/50] (LR: 0.01000) -> Test Accuracy: 89.59%

Epoch [16/50] (LR: 0.01000) -> Test Accuracy: 89.14%

Epoch [17/50] (LR: 0.01000) -> Test Accuracy: 89.81%

Epoch [18/50] (LR: 0.01000) -> Test Accuracy: 89.32%

Epoch [19/50] (LR: 0.01000) -> Test Accuracy: 89.74%

Epoch [20/50] (LR: 0.00100) -> Test Accuracy: 90.25%

Epoch [21/50] (LR: 0.00100) -> Test Accuracy: 92.39%

New Best found: 92.39% (Saved)

Epoch [22/50] (LR: 0.00100) -> Test Accuracy: 92.44%

New Best found: 92.44% (Saved)

Epoch [23/50] (LR: 0.00100) -> Test Accuracy: 92.35%

Epoch [24/50] (LR: 0.00100) -> Test Accuracy: 92.33%

Epoch [25/50] (LR: 0.00100) -> Test Accuracy: 92.42%

Epoch [26/50] (LR: 0.00100) -> Test Accuracy: 92.30%

Epoch [27/50] (LR: 0.00100) -> Test Accuracy: 92.45%

New Best found: 92.45% (Saved)

Epoch [28/50] (LR: 0.00100) -> Test Accuracy: 92.51%

New Best found: 92.51% (Saved)

Epoch [29/50] (LR: 0.00100) -> Test Accuracy: 92.51%

Epoch [30/50] (LR: 0.00100) -> Test Accuracy: 92.55%

New Best found: 92.55% (Saved)

Epoch [31/50] (LR: 0.00100) -> Test Accuracy: 92.69%

New Best found: 92.69% (Saved)

Epoch [32/50] (LR: 0.00100) -> Test Accuracy: 92.57%

Epoch [33/50] (LR: 0.00100) -> Test Accuracy: 92.65%

Epoch [34/50] (LR: 0.00100) -> Test Accuracy: 92.55%

Epoch [35/50] (LR: 0.00100) -> Test Accuracy: 92.66%

Epoch [36/50] (LR: 0.00100) -> Test Accuracy: 92.65%

Epoch [37/50] (LR: 0.00100) -> Test Accuracy: 92.73%

New Best found: 92.73% (Saved)

Epoch [38/50] (LR: 0.00100) -> Test Accuracy: 92.78%

New Best found: 92.78% (Saved)

Epoch [39/50] (LR: 0.00100) -> Test Accuracy: 92.69%

Epoch [40/50] (LR: 0.00010) -> Test Accuracy: 92.62%

```

Epoch [41/50] (LR: 0.00010) -> Test Accuracy: 92.82%
  New Best found: 92.82% (Saved)
Epoch [42/50] (LR: 0.00010) -> Test Accuracy: 92.75%
Epoch [43/50] (LR: 0.00010) -> Test Accuracy: 92.72%
Epoch [44/50] (LR: 0.00010) -> Test Accuracy: 92.68%
Epoch [45/50] (LR: 0.00010) -> Test Accuracy: 92.80%
Epoch [46/50] (LR: 0.00010) -> Test Accuracy: 92.67%
Epoch [47/50] (LR: 0.00010) -> Test Accuracy: 92.91%
  New Best found: 92.91% (Saved)
Epoch [48/50] (LR: 0.00010) -> Test Accuracy: 92.75%
Epoch [49/50] (LR: 0.00010) -> Test Accuracy: 92.88%
Epoch [50/50] (LR: 0.00010) -> Test Accuracy: 92.67%
Advanced Fine-tuning 完成! 最终最佳精度: 92.91%

```

Out[1]: 92.91

```

In [2]: import os, math
import torch
import torch.nn.functional as F

def _u2bin(val: int, bits: int) -> str:
    return format(int(val) & ((1 << bits) - 1), f"0{bits}b")

def _signed_to_hw_unsigned(val: int, bits: int) -> int:
    # match HW: if(temp < 0) temp += 2^bits
    val = int(val)
    if val < 0:
        val += (1 << bits)
    return val

def export_verification_files_hwstyle(
    model,
    data_loader,
    device,
    alpha: float = 1.0,
    bits_a: int = 4,
    bits_w: int = 4,
    bits_out: int = 16,
    stride: int = 1,
    padding: int = 1,                # a_pad padding
    output_dir: str = "verif_files_hwstyle",
    add_bias_to_out: bool = False,   # 如果你确认 HW out 需要加融合 bias,
    force_input_hw: int = 64,        # 关键: 导出时强制输入 64x64 -> 目标层
):
    os.makedirs(output_dir, exist_ok=True)

    model.eval()
    model.to(device)

    # ---- 1) find target conv+bn and capture input to 8x8 conv ----
    conv_8x8, bn_8x8, _ = find_8x8_conv_and_bn(model)

    hook = SaveInput()
    handle = conv_8x8.register_forward_pre_hook(hook)

    images, _ = next(iter(data_loader))

```

```

images = images.to(device)

if force_input_hw is not None:
    images = F.interpolate(images, size=(force_input_hw, force_input_hw),
                           mode="bilinear", align_corners=False)

with torch.no_grad():
    _ = model.features(images)  # 关键: 只跑 features
    handle.remove()

if len(hook.inputs) == 0:
    raise RuntimeError("Hook 没有捕获到目标层输入。")

x_in = hook.inputs[0][0:1].contiguous()  # [1, 8, H, W]
B, C, H, W = x_in.shape
assert C == 8, f"目标层输入通道不是8, 而是{C}"
print(f"[CHECK] x_in shape = {x_in.shape} (ref-like expect [1,8,4,4])")

# ----- 2) fuse conv+bn, apply alpha, quantize to integer -----
w_fused, b_fused = fuse_conv_bn_1x(conv_8x8, bn_8x8)
w_fused = alpha * w_fused

x_int, scale_a = quantize_unsigned(x_in, bits=bits_a)  # [1,8,H,W] in
w_int, scale_w = quantize_signed(w_fused, bits=bits_w)  # [8,8,3,3] in

scale_bias = scale_a * scale_w
b_int = torch.round(b_fused / scale_bias).to(torch.int32)  # [8]

# 统一放 CPU 做 int matmul/acc (MPS 上 int matmul 可能不稳定)
x_int_cpu = x_int.cpu().to(torch.int32)
w_int_cpu = w_int.cpu().to(torch.int32)
b_int_cpu = b_int.cpu().to(torch.int32)

# ----- 3) build a_pad stream -----
a_native = x_int_cpu[0]  # [8, H, W]
a_pad = torch.zeros(C, H + 2 * padding, W + 2 * padding, dtype=torch.int)
a_pad[:, padding:padding + H, padding:padding + W] = a_native
a_pad_flat = a_pad.view(C, -1)  # [8, nijg]
nijg = a_pad_flat.size(1)

a_pad_ni_dim = int(math.sqrt(nijg))
if a_pad_ni_dim * a_pad_ni_dim != nijg:
    raise RuntimeError(f"nijg={nijg} 不是平方数, 无法得到 a_pad_ni_dim")

ki_dim = 3
kijg = range(ki_dim * ki_dim)
o_ni_dim = int((a_pad_ni_dim - (ki_dim - 1) - 1) / stride + 1)
o_nijg = range(o_ni_dim ** 2)

# ----- 4) compute psum[out_col, nijg, kij] -----
w_int_flat = w_int_cpu.view(8, 8, -1)  # [out_col, in_row, kij]
psum = torch.zeros(8, nijg, 9, dtype=torch.int32)

X_stream = a_pad_flat.to(torch.int64)  # [8, nijg]
for kij in kijg:
    Wk = w_int_flat[:, :, kij].to(torch.int64)  # [8,8]

```

```

psum[:, :, kij] = (Wk @ X_stream).to(torch.int32)

# ---- 5) SFP accumulation -> out[out_col, o_nij] ----
out = torch.zeros(8, len(o_nijg), dtype=torch.int32)
for o_nij in o_nijg:
    oy = int(o_nij / o_ni_dim)
    ox = o_nij % o_ni_dim
    for kij in kijg:
        ky = int(kij / ki_dim)
        kx = kij % ki_dim
        in_idx = (oy * a_pad_ni_dim + ox) + (ky * a_pad_ni_dim + kx)
        out[:, o_nij] += psum[:, in_idx, kij]

if add_bias_to_out:
    out += b_int_cpu.view(8, 1)

out_relu = torch.relu(out) # HW reference一般用 ReLU 后的输出

# ---- 6) write activation.txt ----
act_path = os.path.join(output_dir, "activation.txt")
with open(act_path, "w") as f:
    f.write("#time0row7[msb-lsb],time0row6[msb-lst],...,time0row0[msb-lst]\n")
    f.write("#time1row7[msb-lsb],time1row6[msb-lst],...,time1row0[msb-lst]\n")
    f.write("#.....#\n")
    X = a_pad_flat # [8, nijg]
    for t in range(X.size(1)): # time step
        for r in range(X.size(0)): # row
            temp = int(round(X[7 - r, t].item()))
            f.write(_u2bin(temp, bits_a))
        f.write("\n")

# ---- 7) write weight_kij*.txt ----
for kij in kijg:
    w_path = os.path.join(output_dir, f"weight_kij{kij}.txt")
    with open(w_path, "w") as f:
        f.write("#col0row7[msb-lsb],col0row6[msb-lst],...,col0row0[msb-lst]\n")
        f.write("#col1row7[msb-lsb],col1row6[msb-lst],...,col1row0[msb-lst]\n")
        f.write("#.....#\n")
        Wk = w_int_flat[:, :, kij] # [8(out_col), 8(in_row)]
        for col in range(Wk.size(0)): # col0..7
            for row in range(Wk.size(1)): # row7..0
                temp = int(round(Wk[col, 7 - row].item()))
                temp = _signed_to_hw_unsigned(temp, bits_w)
                f.write(_u2bin(temp, bits_w))
            f.write("\n")

# ---- 8) write psum.txt (FINAL OUTPUT, 16 lines when x_in is 4x4) ----
psum_path = os.path.join(output_dir, "psum.txt")
with open(psum_path, "w") as f:
    f.write("#time0col7[msb-lsb],time0col6[msb-lst],...,time0col0[msb-lst]\n")
    f.write("#time1col7[msb-lsb],time1col6[msb-lst],...,time1col0[msb-lst]\n")
    f.write("#.....#\n")
    out_flat = out_relu.view(8, -1) # [8, o_ni_dim*o_ni_dim]
    for t in range(out_flat.size(1)): # time step = o_nij
        for col in range(out_flat.size(0)): # col7..0
            temp = int(round(out_flat[7 - col, t].item()))

```



```

        temp = _signed_to_hw_unsigned(temp, bits_out)
        f.write(_u2bin(temp, bits_out))
    f.write("\n")

# ---- 9) dump scales ----
with open(os.path.join(output_dir, "scales_alpha.txt"), "w") as f:
    f.write(f"alpha_weight_combine = {alpha}\n")
    f.write(f"scale_activation      = {float(scale_a):.8e}\n")
    f.write(f"scale_weight          = {float(scale_w):.8e}\n")
    f.write(f"scale_bias            = {float(scale_bias):.8e}\n")

print("=====")
print(f"[DONE] Generated in: {output_dir}")
print(" - activation.txt")
print(" - weight_kij0..8.txt")
print(" - psum.txt")
print(" - scales_alpha.txt")
print(f" a_pad_dim={a_pad_ni_dim}, nijg={nijg}, out_dim={o_ni_dim}, o_r")
print("=====")

# ===== 直接这样调用 =====
export_verification_files_hwstyle(
    model_part1,
    testloader,
    device,
    alpha=0.8,
    bits_a=4,
    bits_w=4,
    bits_out=16,
    stride=1,
    padding=1,
    output_dir="verif_files_final_hwstyle",
    add_bias_to_out=False,
    force_input_hw=64, # 关键: 保证 activation=36行, psum=16行
)

```

```

[INFO] Found target 8x8 conv at features[40] and BN at features[41].
[CHECK] x_in shape = torch.Size([1, 8, 4, 4]) (ref-like expect [1,8,4,4])
=====
[DONE] Generated in: verif_files_final_hwstyle
- activation.txt
- weight_kij0..8.txt
- psum.txt
- scales_alpha.txt
a_pad_dim=6, nijg=36, out_dim=4, o_nijg=16
=====

```

```

In [2]: import os
import math
import numpy as np

import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision

```

```

import torchvision.transforms as transforms

from models import * # expects QuantConv2d

def get_device() -> torch.device:
    if torch.backends.mps.is_available():
        return torch.device("mps")
    if torch.cuda.is_available():
        return torch.device("cuda")
    return torch.device("cpu")

class VGG16_Part1(nn.Module):
    def __init__(self, num_classes: int = 10):
        super().__init__()
        self.features = nn.Sequential(
            QuantConv2d(3, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64),
            QuantConv2d(64, 64, kernel_size=3, padding=1), nn.BatchNorm2d(64),
            nn.MaxPool2d(kernel_size=2, stride=2),

            QuantConv2d(64, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128),
            QuantConv2d(128, 128, kernel_size=3, padding=1), nn.BatchNorm2d(128),
            nn.MaxPool2d(kernel_size=2, stride=2),

            QuantConv2d(128, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256),
            QuantConv2d(256, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256),
            QuantConv2d(256, 256, kernel_size=3, padding=1), nn.BatchNorm2d(256),
            nn.MaxPool2d(kernel_size=2, stride=2),

            QuantConv2d(256, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512),
            QuantConv2d(512, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512),
            QuantConv2d(512, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512),
            nn.MaxPool2d(kernel_size=2, stride=2),

            QuantConv2d(512, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512),

            QuantConv2d(512, 8, kernel_size=1),
            nn.BatchNorm2d(8),
            nn.ReLU(inplace=True),

            QuantConv2d(8, 8, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(8),
            nn.ReLU(inplace=True),

            QuantConv2d(8, 512, kernel_size=1),
            nn.BatchNorm2d(512),
            nn.ReLU(inplace=True),

            QuantConv2d(512, 512, kernel_size=3, padding=1), nn.BatchNorm2d(512),
            nn.MaxPool2d(kernel_size=2, stride=2),
            nn.AvgPool2d(kernel_size=1, stride=1),
        )
        self.classifier = nn.Linear(512, num_classes)

    def forward(self, x: torch.Tensor) -> torch.Tensor:

```

```

        x = self.features(x)
        x = x.view(x.size(0), -1)
        return self.classifier(x)

def load_pretrained_weights(model: nn.Module, pretrained_path: str, device:
    if not os.path.isfile(pretrained_path):
        print(f"[WARN] checkpoint not found: {pretrained_path}")
        return
    print(f"[INFO] loading checkpoint: {pretrained_path}")
    checkpoint = torch.load(pretrained_path, map_location=device)
    pretrained_dict = checkpoint.get("state_dict", checkpoint)
    model_dict = model.state_dict()
    filtered = {k: v for k, v in pretrained_dict.items() if k in model_dict}
    model_dict.update(filtered)
    model.load_state_dict(model_dict)
    print(f"[INFO] loaded {len(filtered)} tensors")

class SaveInput:
    def __init__(self):
        self.inputs = []

    def __call__(self, module, module_in):
        self.inputs.append(module_in[0].detach())

    def clear(self):
        self.inputs = []

def find_8x8_conv_and_bn(model: nn.Module):
    conv_8x8 = None
    bn_after = None
    conv_idx = None
    for idx, m in enumerate(model.features):
        if isinstance(m, QuantConv2d) and m.in_channels == 8 and m.out_channels == 8:
            conv_8x8 = m
            conv_idx = idx
            if idx + 1 < len(model.features) and isinstance(model.features[idx + 1], nn.BatchNorm2d):
                bn_after = model.features[idx + 1]
            break
    if conv_8x8 is None or bn_after is None:
        raise RuntimeError("Target 8x8 QuantConv2d + following BN(8) not found")
    return conv_8x8, bn_after, conv_idx

def fuse_conv_bn_1x(conv: nn.Conv2d, bn: nn.BatchNorm2d):
    w = conv.weight.detach()
    if conv.bias is None:
        b = torch.zeros(w.size(0), device=w.device, dtype=w.dtype)
    else:
        b = conv.bias.detach()

    running_mean = bn.running_mean.detach()
    running_var = bn.running_var.detach()
    gamma = bn.weight.detach()

```

```

beta = bn.bias.detach()
eps = bn.eps

std = torch.sqrt(running_var + eps)
w_fused = w * (gamma / std).reshape(-1, 1, 1, 1)
b_fused = (b - running_mean) * (gamma / std) + beta
return w_fused, b_fused

def quantize_unsigned(x: torch.Tensor, bits: int):
    qmax = 2**bits - 1
    x_clamped = torch.clamp(x, min=0.0)
    max_val = x_clamped.max().clamp(min=1e-8)
    scale = max_val / qmax
    x_int = torch.round(x_clamped / scale).clamp(0, qmax).to(torch.int32)
    return x_int, scale

def quantize_signed(x: torch.Tensor, bits: int):
    qmax = 2 ** (bits - 1) - 1
    qmin = -2 ** (bits - 1)
    max_abs = x.abs().max().clamp(min=1e-8)
    scale = max_abs / qmax
    x_int = torch.round(x / scale).clamp(qmin, qmax).to(torch.int32)
    return x_int, scale

def _u2bin(val: int, bits: int) -> str:
    return format(int(val) & ((1 << bits) - 1), f"0{bits}b")

def _signed_to_hw_unsigned(val: int, bits: int) -> int:
    val = int(val)
    if val < 0:
        val += (1 << bits)
    return val

def export_verification_files_hwstyle(
    model: nn.Module,
    data_loader,
    device: torch.device,
    alpha: float = 1.0,
    bits_a: int = 4,
    bits_w: int = 4,
    bits_out: int = 16,
    stride: int = 1,
    padding: int = 1,
    output_dir: str = "verif_files_final_hwstyle",
    add_bias_to_out: bool = False,
    force_input_hw: int = 64,
):
    os.makedirs(output_dir, exist_ok=True)

    model.eval()
    model.to(device)

```

```

conv_8x8, bn_8x8, _ = find_8x8_conv_and_bn(model)

hook = SaveInput()
handle = conv_8x8.register_forward_pre_hook(hook)

images, _ = next(iter(data_loader))
images = images.to(device)

if force_input_hw is not None:
    images = F.interpolate(images, size=(force_input_hw, force_input_hw))

with torch.no_grad():
    _ = model.features(images)

handle.remove()

if len(hook.inputs) == 0:
    raise RuntimeError("No input captured by hook.")

x_in = hook.inputs[0][0:1].contiguous() # [1,8,H,W]
_, C, H, W = x_in.shape
if C != 8:
    raise RuntimeError(f"Expected 8 channels at target input, got {C}")

w_fused, b_fused = fuse_conv_bn_1x(conv_8x8, bn_8x8)
w_fused = alpha * w_fused

x_int, scale_a = quantize_unsigned(x_in, bits=bits_a) # [1,8,H,W]
w_int, scale_w = quantize_signed(w_fused, bits=bits_w) # [8,8,3,3]

scale_bias = scale_a * scale_w
b_int = torch.round(b_fused / scale_bias).to(torch.int32) # [8]

x_int_cpu = x_int.cpu().to(torch.int32)
w_int_cpu = w_int.cpu().to(torch.int32)
b_int_cpu = b_int.cpu().to(torch.int32)

a_native = x_int_cpu[0] # [8,H,W]
a_pad = torch.zeros(C, H + 2 * padding, W + 2 * padding, dtype=torch.int)
a_pad[:, padding:padding + H, padding:padding + W] = a_native
a_pad_flat = a_pad.view(C, -1) # [8, nijg]
nijg = a_pad_flat.size(1)

a_pad_ni_dim = int(math.sqrt(nijg))
if a_pad_ni_dim * a_pad_ni_dim != nijg:
    raise RuntimeError(f"nijg={nijg} is not a perfect square")

ki_dim = 3
kijg = range(ki_dim * ki_dim)
o_ni_dim = int((a_pad_ni_dim - (ki_dim - 1) - 1) / stride + 1)
o_nijg = range(o_ni_dim ** 2)

w_int_flat = w_int_cpu.view(8, 8, -1) # [out_col, in_row, kij]

psum = torch.zeros(8, nijg, 9, dtype=torch.int32)

```

```

X_stream = a_pad_flat.to(torch.int64) # [8, nijg]
for kij in kijg:
    Wk = w_int_flat[:, :, kij].to(torch.int64) # [8,8]
    psum[:, :, kij] = (Wk @ X_stream).to(torch.int32)

out = torch.zeros(8, len(o_nijg), dtype=torch.int32)
for o_nij in o_nijg:
    oy = int(o_nij / o_ni_dim)
    ox = o_nij % o_ni_dim
    for kij in kijg:
        ky = int(kij / ki_dim)
        kx = kij % ki_dim
        in_idx = (oy * a_pad_ni_dim + ox) + (ky * a_pad_ni_dim + kx)
        out[:, o_nij] += psum[:, in_idx, kij]

if add_bias_to_out:
    out += b_int_cpu.view(8, 1)

out_relu = torch.relu(out)

act_path = os.path.join(output_dir, "activation.txt")
with open(act_path, "w") as f:
    f.write("#time0row7[msb-lsb],time0row6[msb-lst],...,time0row0[msb-lst]\n")
    f.write("#time1row7[msb-lsb],time1row6[msb-lst],...,time1row0[msb-lst]\n")
    f.write("#.....#\n")
    X = a_pad_flat # [8, nijg]
    for t in range(X.size(1)):
        for r in range(X.size(0)):
            temp = int(X[7 - r, t].item())
            f.write(_u2bin(temp, bits_a))
        f.write("\n")

for kij in kijg:
    w_path = os.path.join(output_dir, f"weight_kij{kij}.txt")
    with open(w_path, "w") as f:
        f.write("#col0row7[msb-lsb],col0row6[msb-lst],...,col0row0[msb-lst]\n")
        f.write("#col1row7[msb-lsb],col1row6[msb-lst],...,col1row0[msb-lst]\n")
        f.write("#.....#\n")
        Wk = w_int_flat[:, :, kij] # [8(out_col), 8(in_row)]
        for col in range(Wk.size(0)):
            for row in range(Wk.size(1)):
                temp = int(Wk[col, 7 - row].item())
                temp = _signed_to_hw_unsigned(temp, bits_w)
                f.write(_u2bin(temp, bits_w))
            f.write("\n")

psum_path = os.path.join(output_dir, "psum.txt")
with open(psum_path, "w") as f:
    f.write("#time0col7[msb-lsb],time0col6[msb-lst],...,time0col0[msb-lst]\n")
    f.write("#time1col7[msb-lsb],time1col6[msb-lst],...,time1col0[msb-lst]\n")
    f.write("#.....#\n")
    out_flat = out_relu.view(8, -1) # [8, o_nijg]
    for t in range(out_flat.size(1)):
        for col in range(out_flat.size(0)):
            temp = int(out_flat[7 - col, t].item())
            temp = _signed_to_hw_unsigned(temp, bits_out)

```

```

        f.write(_u2bin(temp, bits_out))
        f.write("\n")

    with open(os.path.join(output_dir, "scales_alpha.txt"), "w") as f:
        f.write(f"alpha_weight_combine = {alpha}\n")
        f.write(f"scale_activation      = {float(scale_a):.8e}\n")
        f.write(f"scale_weight          = {float(scale_w):.8e}\n")
        f.write(f"scale_bias            = {float(scale_bias):.8e}\n")

    print(f"[DONE] {output_dir}")
    print(f"  x_in: {tuple(x_in.shape)}")
    print(f"  a_pad_dim={a_pad_ni_dim}, nijg={nijg}, out_dim={o_ni_dim}, o_r")
    print(f"  files: activation.txt, weight_kij0..8.txt, psum.txt, scales_alp")

def build_cifar10_loaders(batch_size: int = 128, num_workers: int = 2):
    normalize = transforms.Normalize(mean=[0.491, 0.482, 0.447], std=[0.247,

    train_dataset = torchvision.datasets.CIFAR10(
        root="./data",
        train=True,
        download=True,
        transform=transforms.Compose([
            transforms.RandomCrop(32, padding=4),
            transforms.RandomHorizontalFlip(),
            transforms.ToTensor(),
            normalize,
        ]),
    )

    test_dataset = torchvision.datasets.CIFAR10(
        root="./data",
        train=False,
        download=True,
        transform=transforms.Compose([
            transforms.ToTensor(),
            normalize,
        ]),
    )

    trainloader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size,
    testloader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size,
    return trainloader, testloader

if __name__ == "__main__":
    device = get_device()
    print(f"[Device] {device}")

    model_part1 = VGG16_Part1().to(device)

    PRETRAINED_PATH = "result/VGG16_quant/model_best.pth.tar"
    load_pretrained_weights(model_part1, PRETRAINED_PATH, device)

    _, testloader = build_cifar10_loaders(batch_size=128, num_workers=2)

```

```
export_verification_files_hwstyle(  
    model_part1,  
    testloader,  
    device,  
    alpha=0.8,  
    bits_a=4,  
    bits_w=4,  
    bits_out=16,  
    stride=1,  
    padding=1,  
    output_dir="datafiles",  
    add_bias_to_out=False,  
    force_input_hw=64,  
)
```

[Device] mps

[INFO] loading checkpoint: result/VGG16_quant/model_best.pth.tar

[INFO] loaded 107 tensors

[DONE] datafiles

x_in: (1, 8, 4, 4)

a_pad_dim=6, nijg=36, out_dim=4, o_nijg=16

files: activation.txt, weight_kij0..8.txt, psum.txt, scales_alpha.txt